ASSIGNMENT-2

① Reverse a string using stack

* Algorithm :- (Steps)

1. Start
2. Declare stack and string
3. Read the string from user
4. push each character of the string into stack
5. pop characters from stack one by one
6. Store popped character in another string
7. print reversed string
8. stop

* c-program :-

```
#include <stdio.h>
# include <string.h>
# defination MAX 100
 char stack [MAX];
  int top = -1;
  void push (char x)
 {
  stack [++top] = x;
 }
  char pop( )
 {
  return stack [top--];
 }
  int main( )
 {
  char str[100], rev[100];
  int i;
  printf ("Enter string");
  Scanf ("%s", str);
 for (i=0; i< strlen(str); i++)
     push (str[i]);
```

```
for (i=0 ; i<strlen(str); i++)
    rev[i] = pop();
  rev[i] = '\0';
  printf("Reversed string : %s", rev);
  return 0;
}
```

Example out put :-

input : DATA
Output : ATAD

② Balanced Parentheses :-

* Algorithm :-

1. Start
2. Read Expression
3. Create Empty stack
4. Scan Expression from left to Right
5. If opening bracket ((push into stack)
6. If closing bracket) pop from stack
7. If stack Empty when popping → Not balanced.
8. After scanning whole Expression:
   * If stack Empty → Balanced
   * Else → Not balanced
9. Stop.

* C - program :-

```
# include <stdio.h>
# include <string.h>

  char stack [100];
   int top = -1;

  void push (char x)
  {
   stack [++ top] = x;
  }
```

```
void pop()
{ top--;
}
int main()
{ char exp[100];
  int i;
  printf("Enter Expression");
  scanf("%s", exp);
  for (i=0; i<strlen(exp); i++)
  {
    if (exp[i] == '(')
      push(exp[i]);
    else if (exp[i] == ')')
    {
      if(top==-1)
      {
        print("Not Balanced");
        return 0;
      }
      pop();
    }
  }
  if (top==-1)
    printf("Balanced Expression");
  else
    printf("Not Balanced");
  return 0;
}
```

* Example out put:-

Input : (A+B)*(C-D)

Out put : Balanced Expression.

2. Next Greater Elements using stack

* Alogorithm :-

1. Start
2. Read number of elements and array
3. Push first element to stack
4. Traverse remaining elements.
5. compare current element with stack top.
6. If current element greater → print next greater
7. pop stack element
8. Push current element.
9. If no greater element → print -1
10. stop

* C-program :-

```
# include <stdio.h>
# define MAX100
int stack [MAX];
 int top = 1;
 void Push (int x)
 {
   stack [++top] = x;
 }
   int POP()
 {
  return stack [top--];
 }
 int main ()
 {
   int arr[100], n, i, next;
printf ("Enter number of elements: ");
 scanf ("%d", &n);
 printf ("Enter elements: ");
 for (i=0; i<n; i++)
    Scanf ("%d", &arr[i])
 Push (arr[0]);
  for (i=1; i<n ; i++)
  {
     next = arr[i];
    While (top != -1 && stack[top] < next)
  { printf ("%d -> %d\n", POP(), next);
```

```
}
push(next);
} while (top! = -1)
    printf("%d->-1\n", POP());
  return 0;
}
```

Example output:-

Input: 4 5 2 10 8

output:

4->5
5->10
2->10
10->-1
8->-1

4: Printer Queue simulation:-

* Algorithm:-

1. Start
2. create queue
3. Display menu
   - Add document
   - print document
   - Display queue
4. If add → enqueue document
5. If print → dequeue document
6. If display → show all documents
7. Repeat until exit.

* C-program:-

```
# include <stdio.h>
# define MAX 5
int queue[MAX];
int front =-1, rear=-1;
  void enqueue (int x)
{
  if (rear = = MAX-1)
      Printf("queue full\n");
  else
{ if (front ==-1) front = 0;
```

```
queue[++rear] = x;
}
}
void dequeue()
{
 if (front == -1 || front > rear)
    print ("No document\n");
 else
    print ("printing document %d\n", queue[front++]);
 }
 void display()
{
   int i;
  if (front == -1)
    printf ("Queue Empty\n");
 else
    for (i = front; i <= rear; i++)
      printf ("%d", queue[i]);
 }
 int main()
{
  int ch, x;
   While (1)
{ print ("\n 1. Add document\n 2. Print Document\n 3. Display\n 4.
        Exist\n");
    Scanf ("%d", &ch);

 Switch (ch)
 {
   case 1:
     printf ("Enter document id: ");
     Scanf ("%d", &x);
     enqueue(x);
     break;
 case 2:
     dequeue();
     break;
```

```
case 3:
    display();
    break;
case 4:
    return 0;
}
}
}
```

5: Circular queue operations:-

*Algorithm:-

1. Start
2. initialize front and rear = -1
3. Menu
   - Enqueue
   - Dequeue
   - peek
   - display
4. Enqueue → insert Element
5. Dequeue → Remove front Element
6. Peek → show front Element
7. Display → show all elements
8. stop.

* C-program:-

```
#include <stdio.h>
#define MAX 5
int queue [MAX];
int front = -1 ; rear = -1
Void enqueue (int x)
{
  if((rear+1) % Max == front)
    printf("Queue full \n");
  else
{
  if (front == -1)
      front = 0;
  rear = (rear+1) % MAX;
    queue [rear] = x;
```

```
}
}
void dequeue()
{ if (front==-1)
      front = rear = -1;
  else
      front = (front+1) % MAX;
  }
}
void peak()
{ if (front==-1)
      printf("Queue Empty\n");
  else
      printf("front element: %d\n", queue[front]);
}
void display()
{
  int i=front;
  if (front==-1)
     Printf("Queue Empty\n");
  else
{
  printf("Elements: ");
  While (i!=rear)
{ print("%d", queue[i]);
   i=(i+1)%.MAX;
  }
   printf("%d", queue[rear]);
  }
}
int main()
{
  int ch, x;
  While (1)
{
printf("n1. Enqueue\n2. Dequeue\n3. Peek\n4. Display\n5. Exit\n");
Scanf("%d", &ch);
```

```
Switch (ch)
{ case 1:
    printf("Enter Elemen H:");
    Scanf ("%d", &x);
    Enaueue(x);
    break;
  case 2:
      deaueue();
      break;
  case 3:
        peek();
        break;
  case 4:
        display();
        break;
  case 5:
        return 0;
  }
}
}
```